

Battle Jamón：微服務擂台

來源：<https://codelabs.developers.google.com/codelabs/battle-jamon?hl=zh-tw>

步驟數：7

在本程式碼研究室中，您將建構微服務，在競技場中互相「丟擲」果醬，藉此對抗其他微服務。

目錄

1. [1. 簡介](#)
2. [2. 登入 Google Cloud](#)
3. [3. 部署微服務](#)
4. [4. 要求將應用程式納入競技場](#)
5. [5. 進行及部署變更](#)
6. [6. 恭喜](#)
7. [7. 常見問題](#)

1. 簡介

上次更新時間：2020 年 5 月 5 日

請務必先詳閱《[條款及細則](#)》再繼續操作。

微服務戰鬥競技場

你是否曾參與雪仗，在移動的同時，向其他人丟雪球？如果還沒用過，不妨找機會試試！但現在您不必擔心被揍，可以建構小型網路存取服務 (微服務)，與其他微服務展開史詩般的戰鬥。由於我們是在 Spring I/O 舉辦這場微服務大戰，因此微服務會投擲西班牙火腿，而不是雪球。

你可能想知道... 但微服務如何「將」哈蒙火腿「丟」給其他微服務？微服務可以接收網路要求 (通常是透過 HTTP)，並傳回回應。系統會提供「競技場管理員」，將競技場的目前狀態傳送給微服務，然後微服務會傳回指令，指定要執行的動作。

當然，目標是贏得勝利，但過程中您會瞭解如何在 Google Cloud 上建構及部署微服務。

運作方式

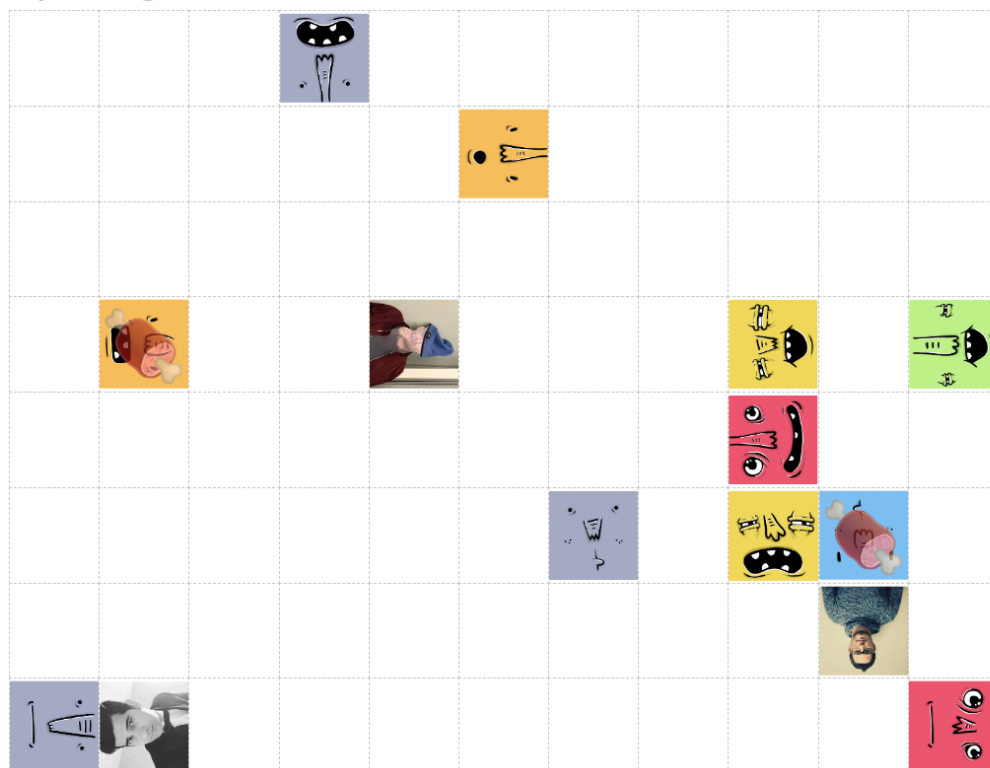
您可以使用任何技術建構微服務 (或從 Java、Kotlin 或 Scala 啟動器中選擇)，然後在 Google Cloud 上部署微服務。部署完成後，請填寫表單，告知我們微服務的網址，我們會將其加入競技場。

競技場包含特定戰鬥的所有玩家。Spring I/O Bridge 會議將有專屬場地。每位玩家代表一項微服務，會四處移動並向其他玩家投擲哈蒙。

競技場管理工具大約每秒會呼叫一次微服務，傳送目前的競技場狀態 (玩家所在位置)，而微服務會回覆要執行的指令。在競技場中，您可以向前移動、向左或向右轉，或是投擲火腿。投擲的西班牙火腿會朝玩家面向的方向移動最多三格。如果火腿「擊中」其他玩家，投擲者可得一分，被擊中的玩家則會扣一分。系統會根據目前的玩家人數自動調整競技場大小。

以下是競技場的樣子，其中有三位虛構玩家：

Spring I/O - Battle Jamón



High Scores 🏆

	judomu	11ms	14
	Prashant Bhuru	39ms	13
	Daniel	16ms	11
	Anbu	11ms	2
	Stefano	11ms	1
	DomenP	119ms	0
	Antoni	14ms	-1
	Hatim	14ms	-2
	David	12ms	-4
	Geostyl	10ms	-5
	Sewerin	11ms	-6
	Simone	17ms	-6
	Denniss	11ms	-8
	Ladislav	37ms	-9

Battle Jamón 競技場範例

循環衝突

在競技場中，多位玩家可能會嘗試執行衝突的動作。舉例來說，兩位玩家可能會嘗試移動到同一個空間。如有衝突，回應時間最短的微服務會勝出。

觀看對戰

如要查看微服務在戰鬥中的表現，請[前往即時競技場](#)！

Battle API

如要與競技場管理員合作，微服務必須實作特定 API，才能參與競技場。競技場管理員會透過 HTTP POST 將目前的競技場狀態傳送至您提供的網址，JSON 結構如下：

```
{
  "_links": {
    "self": {
      "href": "https://YOUR_SERVICE_URL"
    }
  },
  "arena": {
    "dims": [4,3], // width, height
    "state": {
      "https://A_PLAYERS_URL": {
        "x": 0, // zero-based x position, where 0 = left
        "y": 0, // zero-based y position, where 0 = top
        "direction": "N", // N = North, W = West, S = South, E = East
        "wasHit": false,
        "score": 0
      }
      ... // also you and the other players
    }
  }
}
```

HTTP 回應必須是狀態碼 200 (OK)，且回應內文包含您的下一步行動，並編碼為下列其中一個大寫字元：

```
F <- move Forward
R <- turn Right
L <- turn Left
T <- Throw
```

就是這麼簡單！接下來，我們將逐步說明如何在 [Cloud Run](#) 上部署微服務。Cloud Run 是 Google Cloud 服務，可執行微服務和其他應用程式。

步驟 2 · 約 5 分鐘

2. 登入 Google Cloud

如要在 Cloud Run 上部署微服務，您必須登入 Google Cloud。我們會將抵免額套用至你的帳戶，你不需要輸入信用卡資訊。使用個人帳戶 (例如 gmail.com) 通常比使用 G Suite 帳戶更不容易發生問題，因為有時 G Suite 管理員會禁止使用者使用特定 Google Cloud 功能。此外，我們使用的網頁控制台應可順暢搭配 Chrome 或 Firefox 運作，但可能無法在 Safari 中正常運作。

3. 部署微服務

只要微服務可公開存取並符合 Battle API 規定，您就能使用任何技術建構微服務，並部署至任何位置。不過，為了簡化流程，我們會協助您從範例服務著手，並將其部署至 Cloud Run。

選取要開始使用的範例

您可以從以下兩種戰鬥微服務範例著手：

Java 和 Spring Boot	資料來源	在 Cloud Run 上部署
Kotlin 和 Spring Boot	資料來源	在 Cloud Run 上部署

請注意，部署時可能會發生錯誤：

```
Error: the service did not become ready in 30s, check Cloud Console for logs to see why it failed
```

別擔心，這只是因為[逾時時間太短](#)。如要取得服務的網址，請先在 Cloud Shell 中執行下列指令，取得 GCP 專案：

```
gcloud projects list
```

現在請取得專案 ID，然後執行下列指令，將 `YOUR_PROJECT_ID` 改成您的專案 ID：

```
export PROJECT_ID=YOUR_PROJECT_ID
```

現在列出 Cloud Run 服務：

```
gcloud run services list --platform=managed --project=$PROJECT_ID
```

請照常完成本程式碼研究室的其餘部分，並使用列印清單中的服務網址。

決定要從哪個範例開始後，請點選上方的「Deploy on Cloud Run」（部署至 Cloud Run）按鈕。系統會啟動 [Cloud Shell](#)（雲端虛擬機器的網頁版控制台），並在其中複製來源、建構可部署的套件（Docker 容器映像檔），然後上傳至 [Google Container Registry](#)，最後部署到 [Cloud Run](#)。

系統詢問時，請指定 `us-central1` 區域。

下方的螢幕截圖顯示微服務建構和部署的 *Cloud Shell* 輸出內容

```

t [PROJECT_ID]"
demo@cloudshell:~$ cloudshell_open --repo_url "https://github.com/jamesward/hello-netcat.git" --page "editor"
[ ✓ ] Cloned git repository https://github.com/jamesward/hello-netcat.git.
[ ✓ ] Queried list of your GCP projects
[ ? ] Would you like to use existing GCP project jw-demo to deploy this app? Yes
[ ✓ ] Enabled Cloud Run API on project jw-demo.
[ ? ] Choose a region to deploy this application: us-central1
[ ! ] Attempting to build this application with its Dockerfile...
[ ! ] FYI, running the following command:
      docker build -t gcr.io/jw-demo/hello-netcat .
[ ✓ ] Built container image gcr.io/jw-demo/hello-netcat
[ ! ] FYI, running the following command:
      docker push gcr.io/jw-demo/hello-netcat
[ ✓ ] Pushed container image to Google Container Registry.
[ ! ] FYI, running the following command:
      gcloud run deploy hello-netcat\
        --project=jw-demo\
        --platform=managed\
        --region=us-central1\
        --image=gcr.io/jw-demo/hello-netcat\
        --memory=512Mi\
        --allow-unauthenticated
[ ✓ ] Successfully deployed service hello-netcat to Cloud Run.
* This application is billed only when it's handling requests.
* Manage this application at Cloud Console:
  https://console.cloud.google.com/run?project=jw-demo
* Learn more about Cloud Run:
  https://cloud.google.com/run/docs
[ ✓ ] Your application is now live here:
      https://hello-netcat-weurlhfjmq-uc.a.run.app
demo@cloudshell:~$ █

```

確認微服務是否正常運作

在 Cloud Shell 中，您可以向新部署的微服務提出要求，並將 `YOUR_SERVICE_URL` 替換為服務的網址 (位於 Cloud Shell 中「Your application is now live here」這一行之後)：


```
curl -d '{
  "_links": {
    "self": {
      "href": "https://foo.com"
    }
  },
  "arena": {
    "dims": [4,3],
    "state": {
      "https://foo.com": {
        "x": 0,
        "y": 0,
        "direction": "N",
        "wasHit": false,
        "score": 0
      }
    }
  }
}' -H "Content-Type: application/json" -X POST -w "\n" \
https://YOUR_SERVICE_URL
```

您應該會看到 F、L、R 或 T 的回應字串。

步驟 4 · 約 15 分鐘

4. 要求將應用程式納入競技場

如要加入競技場，請傳送訊息至 [#battle-jamon 頻道](#)，並附上您的姓名、Cloud Run 服務網址，以及 GitHub 使用者名稱 (選填，用於顯示個人資料相片 / 大頭照)。我們驗證資訊後，你的球員就會出現在 [競技場](#)。

5. 進行及部署變更

如要進行變更，您必須在 Cloud Shell 中設定 GCP 專案和所用範例的相關資訊。首先列出您的 GCP 專案：

```
gcloud projects list
```

您可能只有一個專案。從第一欄複製 `PROJECT_ID`，然後貼到下列指令中 (將 `YOUR_PROJECT_ID` 替換為實際的專案 ID)，以便設定環境變數，供後續指令使用：

```
export PROJECT_ID=YOUR_PROJECT_ID
```

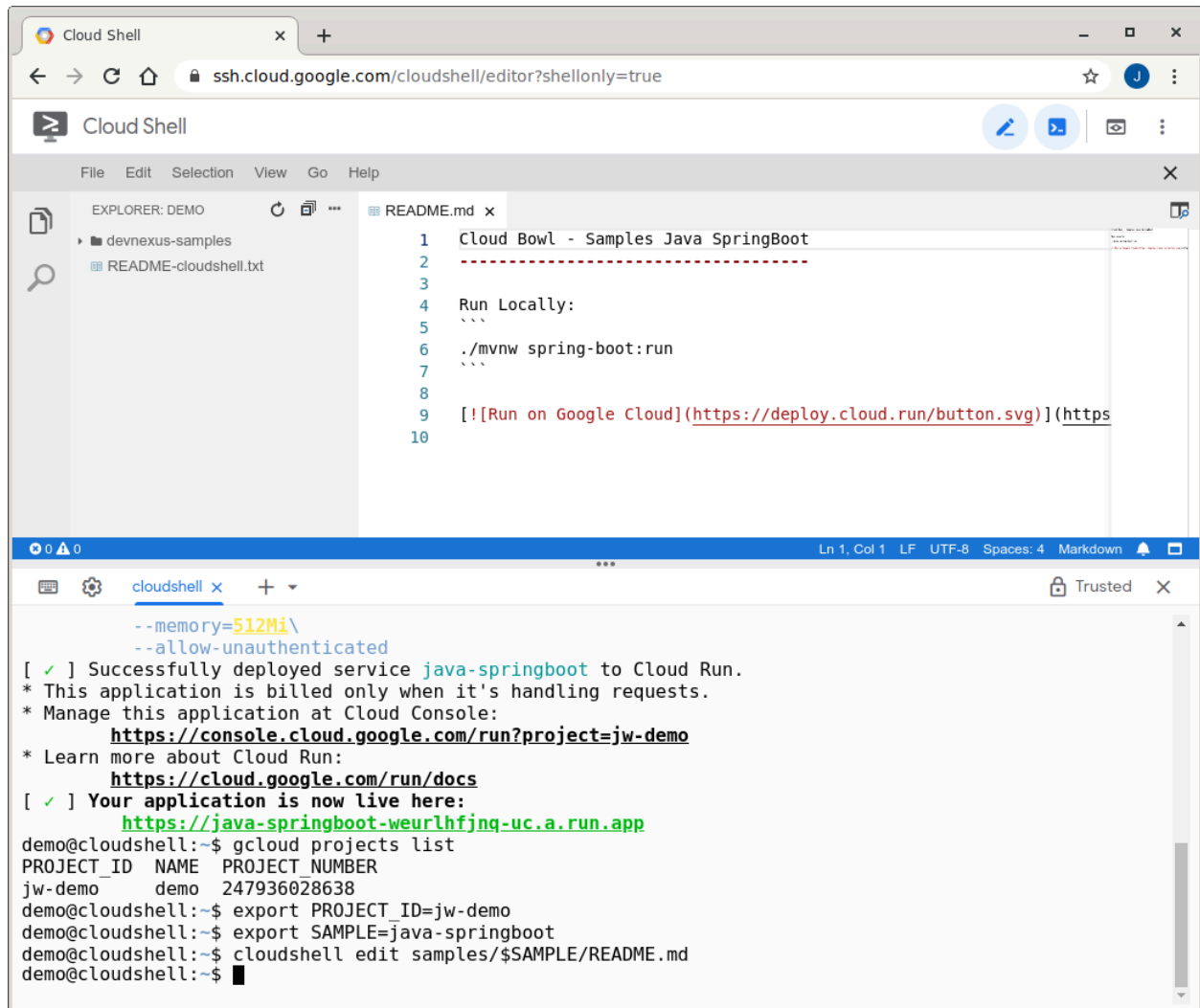
現在請為您使用的範例設定另一個環境變數，以便在後續指令中指定正確的目錄和服務名稱：

```
# Copy and paste ONLY ONE of these
export SAMPLE=java-springboot
export SAMPLE=kotlin-springboot
```

現在，您可以在 Cloud Shell 中編輯微服務的來源。如要開啟 Cloud Shell 網頁版編輯器，請執行下列指令：

```
cloudshell edit cloudbowl-microservice-game/samples/$SAMPLE/README.md
```

接著，系統會顯示變更的進一步操作說明。



```
Cloud Shell
ssh.cloud.google.com/cloudshell/editor?shellonly=true

Cloud Shell
File Edit Selection View Go Help

EXPLORER: DEMO
devnexus-samples
README-cloudshell.txt

README.md x
1 Cloud Bowl - Samples Java SpringBoot
2 -----
3
4 Run Locally:
5 ```
6 ./mvnw spring-boot:run
7 ```
8
9 [!Run on Google Cloud](https://deploy.cloud.run/button.svg)(https
10

Ln 1, Col 1 LF UTF-8 Spaces: 4 Markdown

cloudshell x Trusted x
--memory=512Mi\
--allow-unauthenticated
[ ✓ ] Successfully deployed service java-springboot to Cloud Run.
* This application is billed only when it's handling requests.
* Manage this application at Cloud Console:
https://console.cloud.google.com/run?project=jw-demo
* Learn more about Cloud Run:
https://cloud.google.com/run/docs
[ ✓ ] Your application is now live here:
https://java-springboot-weurlhfjng-uc.a.run.app
demo@cloudshell:~$ gcloud projects list
PROJECT_ID NAME PROJECT_NUMBER
jw-demo demo 247936028638
demo@cloudshell:~$ export PROJECT_ID=jw-demo
demo@cloudshell:~$ export SAMPLE=java-springboot
demo@cloudshell:~$ cloudshell edit samples/$SAMPLE/README.md
demo@cloudshell:~$
```

Cloud Shell 編輯器已開啟範例專案

儲存變更後，在 Cloud Shell 中啟動應用程式：

```
cd cloudbowl-microservice-game/samples/$SAMPLE
./mvnw spring-boot:run
```

應用程式執行後，開啟新的 Cloud Shell 分頁，並使用 curl 測試服務：

```
curl -d '{
  "_links": {
    "self": {
      "href": "https://foo.com"
    }
  },
  "arena": {
    "dims": [4,3],
    "state": {
      "https://foo.com": {
        "x": 0,
        "y": 0,
        "direction": "N",
        "wasHit": false,
        "score": 0
      }
    }
  }
}' -H "Content-Type: application/json" -X POST -w "%n" \
http://localhost:8080
```

準備好部署變更時，請使用 `pack` 指令在 Cloud Shell 中建構專案。這項指令會使用建構套件偵測專案類型、編譯專案，並建立可部署的構件 (Docker 容器映像檔)。

```
pack build gcr.io/$PROJECT_ID/$SAMPLE \
--path ~/cloudbowl-microservice-game/samples/$SAMPLE \
--builder gcr.io/buildpacks/builder
```

容器映像檔建立完成後，請使用 `docker` 指令 (在 Cloud Shell 中) 將容器映像檔推送至 Google Container Registry，以便 Cloud Run 存取：

```
docker push gcr.io/$PROJECT_ID/$SAMPLE
```

現在將新版本部署至 Cloud Run：

```
gcloud run deploy $SAMPLE\
  --project=$PROJECT_ID\
  --platform=managed\
  --region=us-central1\
  --image=gcr.io/$PROJECT_ID/$SAMPLE\
  --memory=512Mi\
  --allow-unauthenticated
```

現在競技場就會使用新版本！

步驟 6 · 約 0 分鐘

6. 恭喜

恭喜！您已成功建構及部署微服務，可與其他微服務對戰！祝您好運！

繼續學習

- [更多 Spring 程式碼研究室](#)

參考文件

- [Cloud Run 說明文件](#)
-

7. 常見問題

為什麼我的微服務沒有顯示在競技場中？

將新機器人加入競技場需要手動操作，因此最多可能需要 15 分鐘。您可以在支援設定檔中查看服務：
<https://github.com/cloudbowl/arenas/blob/master/springio.json>

最終戰鬥的運作方式為何？

最終戰開始時，分數會重設。戰鬥時間為 5 分鐘，之後系統會記錄分數，以決定獎勵。

決戰前競技場的運作方式為何？

在最終戰鬥前一天，每當有新玩家加入競技場，分數就會重設。此外，我們可能會手動重設分數幾次，讓玩家在全新狀態下，更清楚瞭解機器人的表現。

如何獲勝？

部署新版演算法 (即讓機器人變得更聰明) 時，您可以觀察競技場，瞭解變更的成效。繼續調整，並視需要部署新版本。

我可以進行本機開發嗎？

太棒了！首先，在 Cloud Shell 中將範例壓縮成 ZIP 檔案：

```
cd ~/cloudbowl-microservice-game/samples; zip -r cloudbowl-sample.zip $SAMPLE
```

然後在 Cloud Shell 中，將 zip 檔案下載到電腦：

```
cloudshell download-file cloudbowl-sample.zip
```

現在請在本機上完成其餘步驟...

解壓縮檔案，然後進行及測試變更。如要部署變更，請[安裝 gcloud CLI](#)，然後登入 Google Cloud：

```
gcloud auth login
```

將環境變數 `PROJECT_ID` 和 `SAMPLE` 設為與 Cloud Shell 相同的值。

然後使用 Cloud Build 建構容器 (從根專案目錄)：

```
gcloud alpha builds submit . --pack=image=gcr.io/$PROJECT_ID/$SAMPLE
```

現在部署新容器：

```
gcloud run deploy $SAMPLE --project=$PROJECT_ID --platform=managed --region=us-central1 --  
image=gcr.io/$PROJECT_ID/$SAMPLE --memory=512Mi --allow-unauthenticated
```
